Routing

Volume 9

1 Background and Task

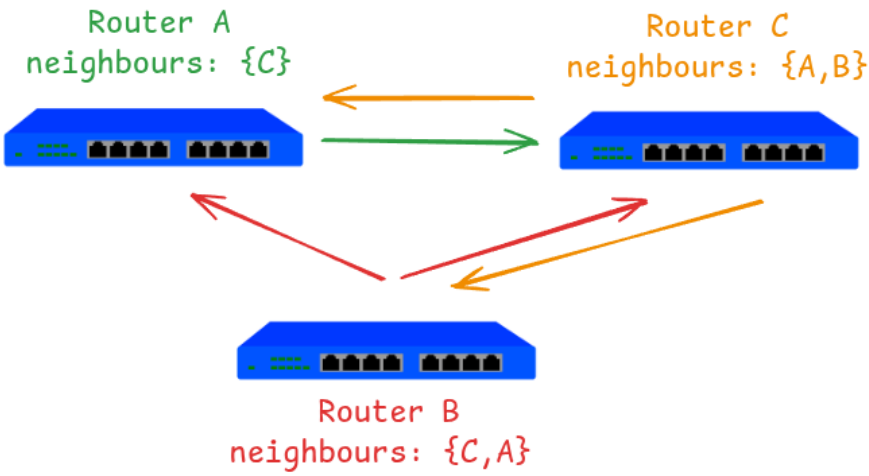


Figure 1: Example network of routers

Consider a network of "routers" that can send "packets" of information to each other. Each router (e.g., some router i) has a list of *neighbours*, which is an array of routers that i can send packets to. The image above shows a network with 3 routers: `A`, `B`, and `C`, and assume that an arrow going from some Router i to Router j means that i considers j to be a neighbour, and i can therefore send packets to j.
Let's say that:

1. Router `A`'s first and only neighbour was `C` (note only one arrow coming out of A).
2. Router `B`'s first neighbour was `C`, and second neighbour was `A`.
3. Router `C`'s first neighbour was `A`, and second neighbour was `B`.

The order of neighbours isn't shown in the diagram (only via the array of neighbours)

Your Task: Given a struct representing the **starting router** of a packet, an array of `size_t` values representing a **route** taken by the packet, and a `size_t` indicating the length of the array, **print out the names of the routers along the route of the packet** (including the one it started at).

What is a a route? Let's say we were told that the packet started from router `A`, and that the packet reported taking a **route** (array) with the values {0, 1, 0}. This means that the packet started at `A`, then went to router `C` (the neighbour at index 0), then went to router `B` (which is the neighbour at index 1 for router `C`) then it went back to router `C` (which is the neighbour at index 0 for router `B`).

2 Code

You are given the following empty function(s) in `student.c` which you should implement. Only make changes within `student.c`.

```
void trace_route(Router current_router, size_t *route_path,
                    size_t path_length){

}
```

The details about the `Router` struct are as follows:

```
typedef struct Router {
    char *name;
    struct Router **neighbour_list;
    size_t num_neighbours;
} Router;
```

`name` is a null-terminated string that represents the name of a router. `neighbour_list` is an array of `Router *`. This means that `neighbour_list[i]` gives you a **pointer** to the ith neighbour of the current router. The length of `neighbour_list` is given by `num_neighbours`.

To compile your program, run `make`, to run your program run `./routing`.
To specify the entire computer network, in general, the input format is given as:

```
[number of routers]
<router names, each router name on its own line>
[router name]:[name(s) of the neighbour routers,
                                        separated by commas]
;
[the route path as integers, separated by commas]
```

For example, consider the following input (**representing Figure 1 above**):

```
3
A
B
C
A:C
B:C,A
C:A,B
;
0,1,0
```

This input says we have 3 routers. The first router named A, the second router named B, and the third named C. **The packet will always start at the first router given, in this case it's A.**

1. A's first neighbour is C.
2. B's first neighbour is C, and second neighbour is A.
3. C's first neighbour is A, and second neighbour is B.

Lastly, the path taken by the packet is the 0th neighbour, then the 1th neighbour, then the 0th neighbour. **We will always give you a valid route**.
The output (which ends on a newline character) of the program is then:

```
@A->@C->@B->@C
```

Because it started on A, then took the first neighbour of A, which is C. It then took the second neighbour of C, which is B. It then took the first neighbour of B, which is C.
More examples in the next section.

3 Sample Input

Note: Input handling has been done for you. We own all Routers, we will free them. Just implement the required function(s) (which means just output/printing).

Please end your output on a newline character.
Sample Input 1

```
2
A
B
A:B
;
0
```

Expected Output 1

```
@A->@B
```

Sample Input 2

```
2
A
B
A:B
B:A
;
0,0
```

Expected Output 2

```
@A->@B->@A
```

Sample Input 3

```
3
A
B
C
A:B
B:C
C:A
;
0,0,0,0
```

Expected Output 3

```
@A->@B->@C->@A->@B
```

Sample Input 4

```
4
us-east-1
us-west-2
eu-east-3
eu-west-4
us-east-1:eu-west-4
eu-east-3:us-east-1,us-west-2
us-west-2:us-east-1,us-west-2,eu-east-3
eu-west-4:us-east-1,us-west-2,eu-east-3
;
0,2,1,2
```

Expected Output 4

```
@us-east-1->@eu-west-4->@eu-east-3->@us-west-2->@eu-east-3
```
